



LangChain



LangChain

What is LangChain ?

*is a framework for building applications
powered by language models.*

Why Was LangChain Created?

1) Regular Code – Connect to LLMs Directly

You can call each model with its own library / API.



```
from openai import OpenAI
client = OpenAI(api_key="YOUR_API_KEY")
response = client.chat.completions.create(
    model="gpt-4o",
    messages=[{"role": "user", "content": "Hello!"}]
)
print(response.choices[0].message.content)
```



```
import google.generativeai as genai
genai.configure(api_key="YOUR_API_KEY")
model = genai.GenerativeModel("gemini-1.5-flash")
response = model.generate_content("Hello!")
print(response.text)
```



```
from groq import Groq
client = Groq(api_key="YOUR_API_KEY")
response = client.chat.completions.create(
    model="llama-3.3-70b-versatile",
    messages=[{"role": "user", "content": "Hello!"}]
)
print(response.choices[0].message.content)
```

2) Using LangChain – One Simple Interface

LangChain gives a common, easy way to use different LLMs.

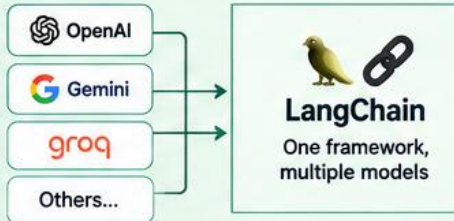
```
from langchain.chat_models import ChatOpenAI
# You can also use ChatGoogleGenerativeAI or ChatGroq

llm = ChatOpenAI(
    model="gpt-4o",
    temperature=0.2
)

from langchain.schema import HumanMessage

response = llm.invoke([
    HumanMessage(content="Hello! How are you?")
])

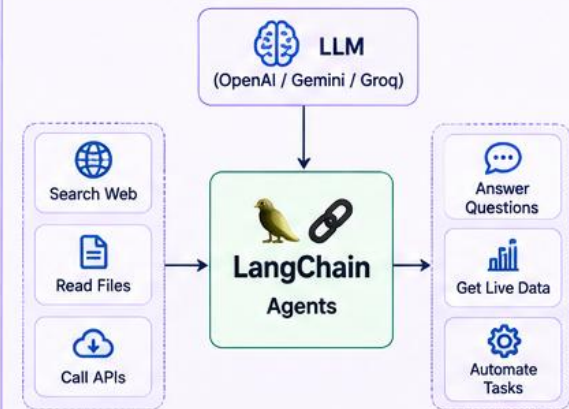
print(response.content)
```



Use the
same code style
for all models!

3) How LangChain Helps in Building Agents

LangChain provides ready-to-use building blocks to create smart AI Agents that can think and take actions.



What LangChain gives you for Agents?

- ✓ Tool calling (search, APIs, databases, calculators, etc.)
- ✓ Memory (remembers previous messages)
- ✓ Chains & Workflows (step-by-step AI processing)
- ✓ Integration with vector databases for RAG (document Q&A)
- ✓ Easy way to build multi-step, autonomous AI Agents

❌ Disadvantages of Connecting Models Without LangChain

- 😞 Different code for each model (OpenAI, Gemini, Groq, etc.)
- 😞 No ready-made support for memory, tools, retrieval (RAG), or agents
- 😞 You have to build chains and workflows manually
- 😞 Harder to switch models or scale the application
- 😞 More time and code to handle logs, streaming, prompt templates, etc.

✅ Advantages of Using LangChain

- ✓ One common, simple interface for many LLMs
- ✓ Ready components for chains, memory, RAG and agents
- ✓ Easier to build complex, multi-step AI applications
- ✓ Switch models anytime with minimal code changes
- ✓ Saves development time – focus on your idea, not low-level code

💡 Key Takeaways

- 🚀 Use direct API calls for small, simple tasks.
- 🤖 Use LangChain when you want to build real AI applications like chatbots, document Q&A or autonomous Agents.
- 🎯 LangChain makes AI development faster, easier and more powerful – even for beginners!

Why Was LangChain Created?

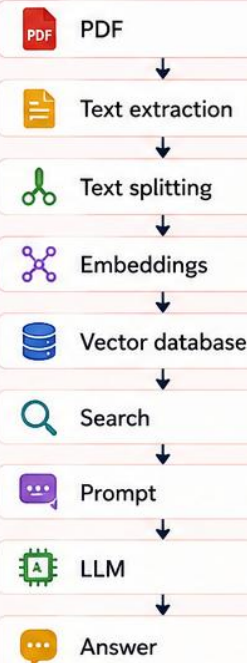
Suppose you want:
“Build a chatbot that reads a PDF
and answers questions.”



Problems:

- ✗ Many steps
- ✗ Different libraries
- ✗ Lots of boilerplate code
- ✗ Hard to maintain
- ✗ Error handling everywhere
- ✗ Difficult to scale

You need to handle:

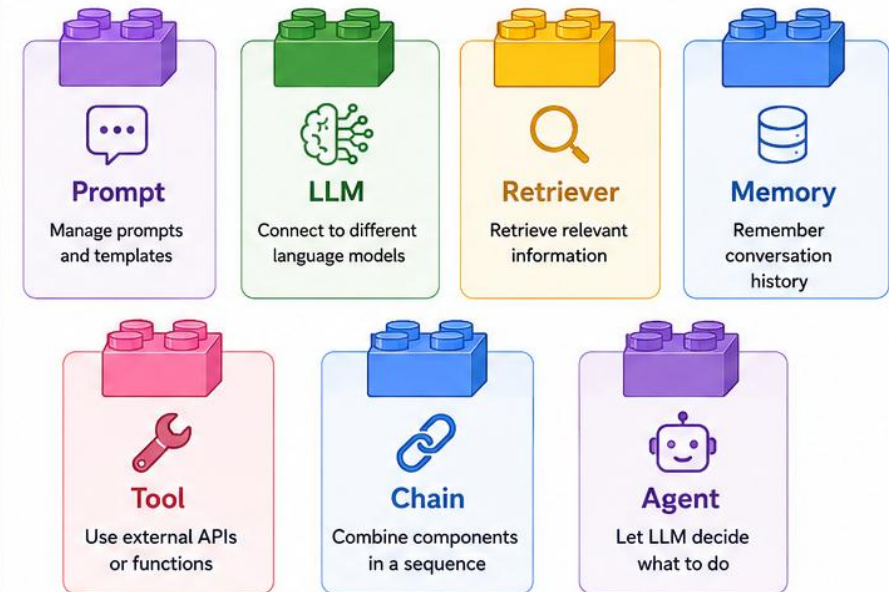


That's a lot of code.
Time-consuming and complex!



Think of LangChain like **LEGO**.

Instead of building every component from scratch:

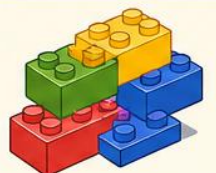


You connect components together.

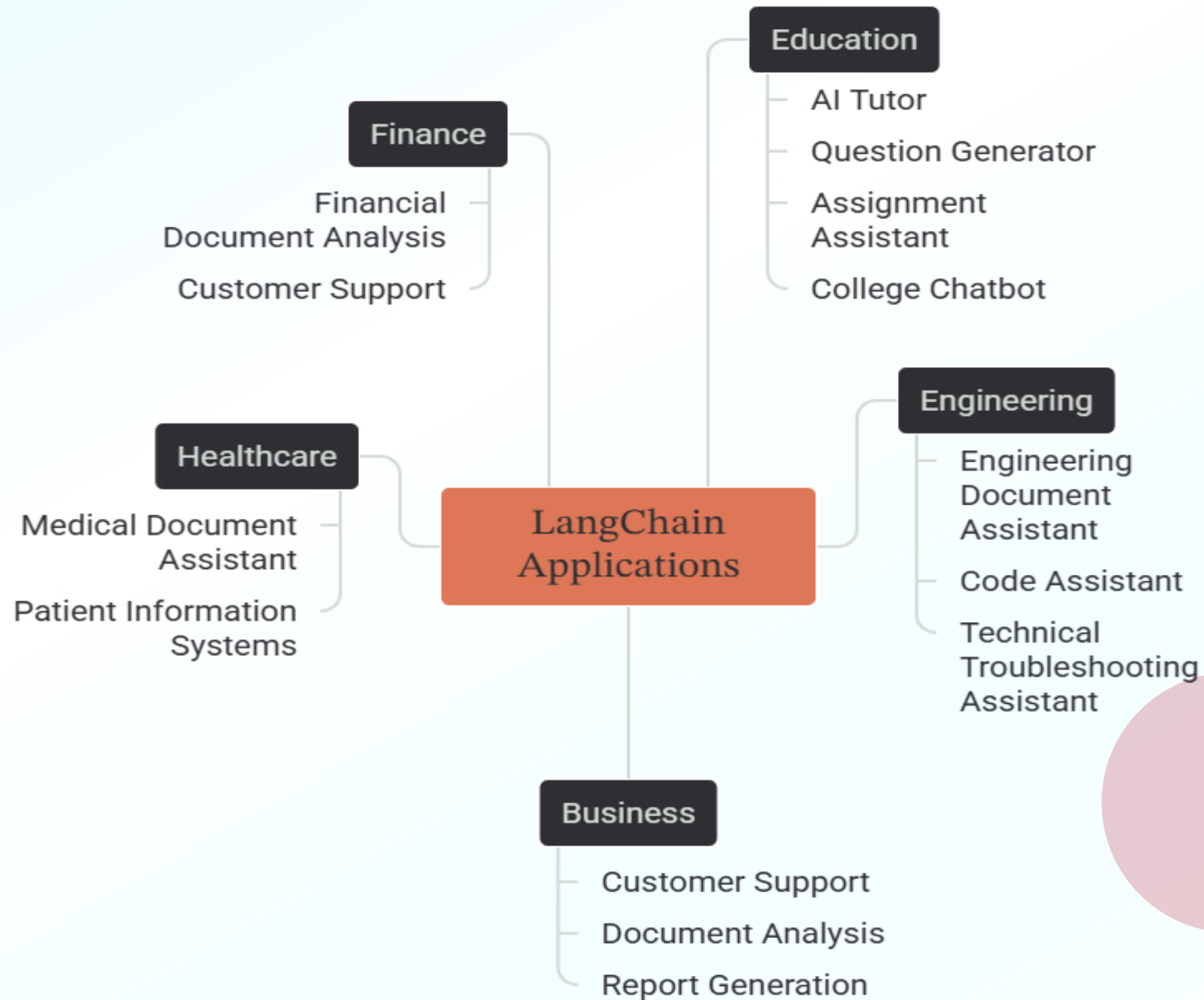


Key analogy

“LangChain is like **LEGO** for building LLM applications.”



LangChain Applications Across Industries



Installation

```
pip install -U langchain
```

```
pip install -U langchain-google-genai
```

LangChain Chains: Sequential vs Parallel

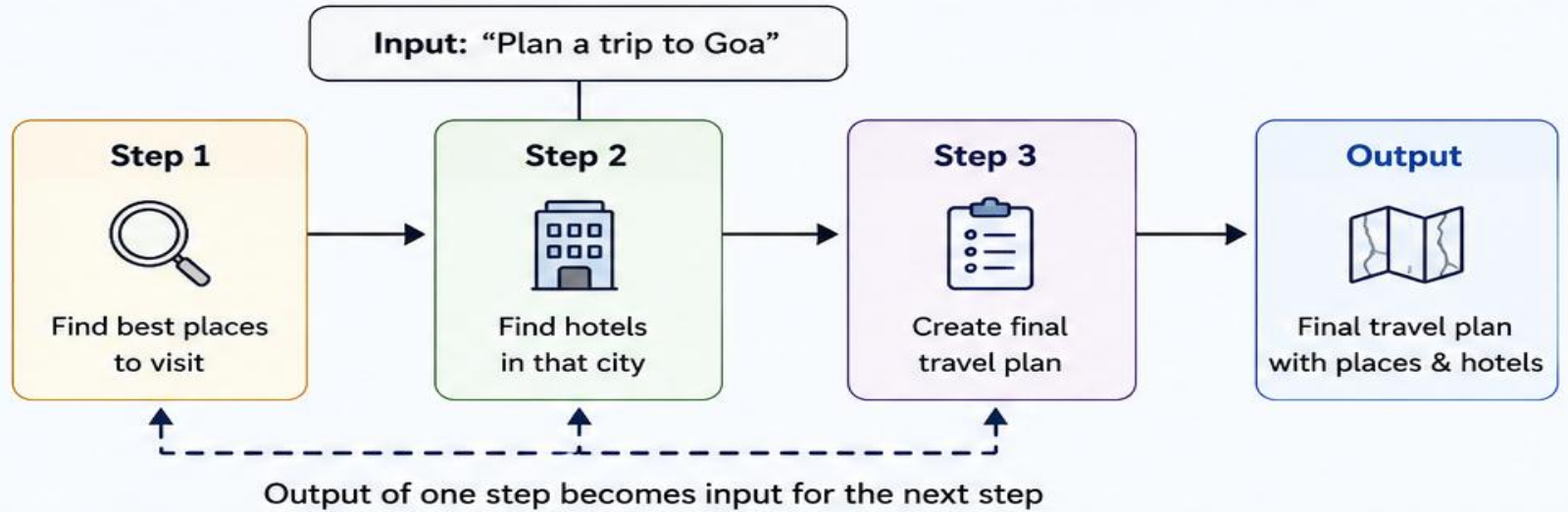
1. Sequential Chain

Steps are executed **one after another**.

Example

You plan a trip from your city to another city.

- 1 Find the best places to visit
- 2 Find the hotels in that city
- 3 Create the final travel plan



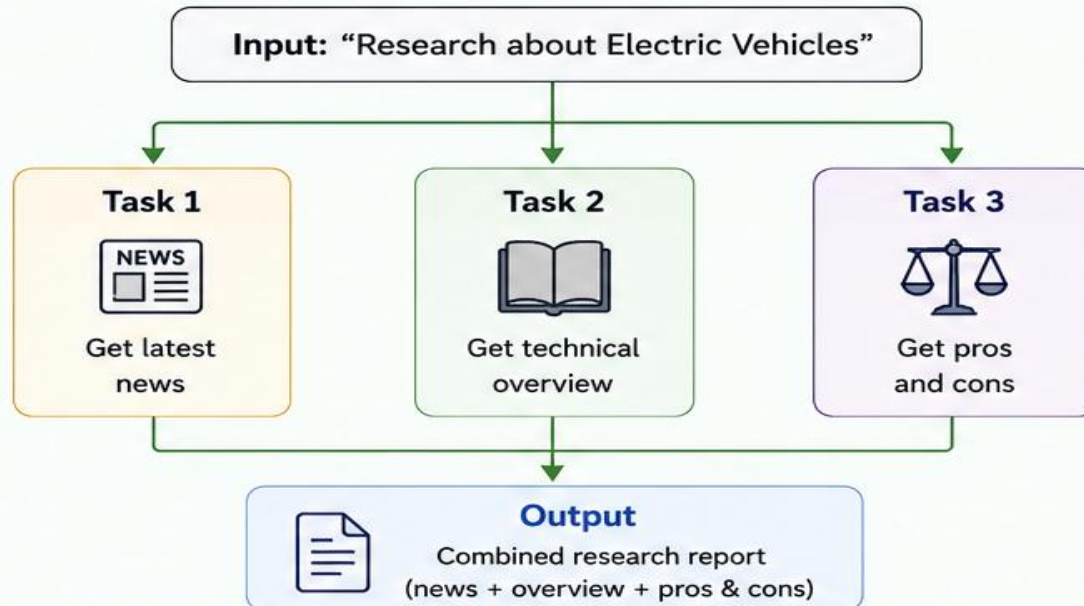
2. Parallel Chain

Steps are executed **at the same time**.

Example

You want to research a topic ("Electric Vehicles").

- 1 Get latest news
- 2 Get technical overview
- 3 Get pros and cons



Key Difference

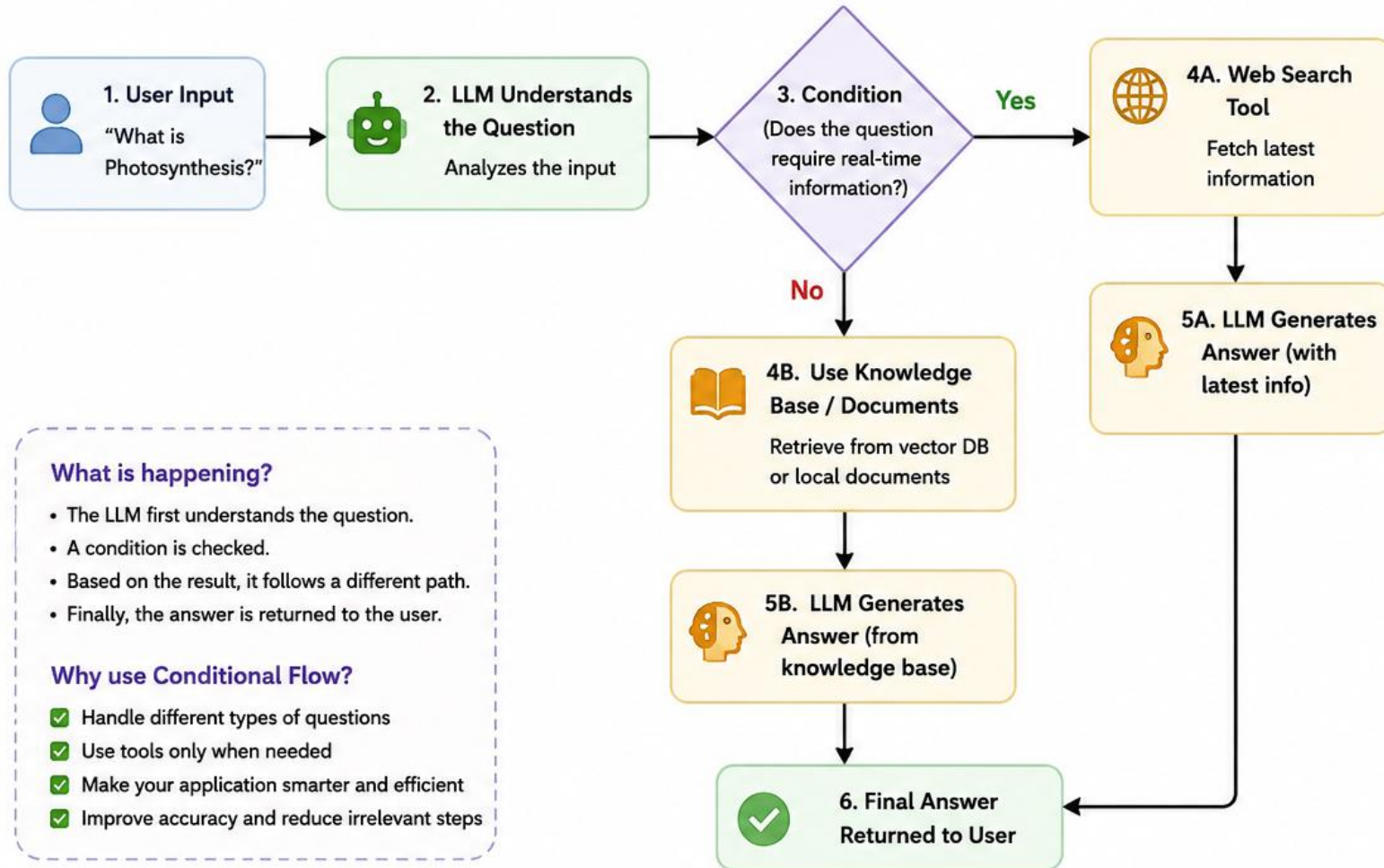
Sequential = One after another

Parallel = At the same time



Conditional LangChain Flow

The chain takes a decision and follows a different path based on a condition.



Example

Example 1:

User:

"What is the current temperature in Hyderabad?"

Flow:

User Input → LLM Understands → Condition (Yes) → Web Search → LLM Generates Answer → Output

Why?

It needs real-time information, so it uses Web Search.

Example 2:

User:

"What is Photosynthesis?"

Flow:

User Input → LLM Understands → Condition (No) → Knowledge Base → LLM Generates Answer → Output

Why?

It is a general knowledge question, so it uses stored documents.



In LangChain, this is implemented using:

```
RunnableBranch(  
    (condition_function, chain_if_true),  
    (lambda x: True, chain_if_false)  
)
```

This allows your application to be dynamic, context-aware, and more powerful.